

Колоночное хранение

Table of contents

Колоночное хранение	1
Когда строковая раскладка мешает	1
Как выглядит колоночная раскладка	1
Почему колонки хорошо сжимаются	3
Цена записи	3
OLTP vs OLAP	4
Пример: маркетплейс	4

Колоночное хранение

В модуле про базовые структуры мы разобрали строковое хранение: `heap` и `clustered` держат **всю строку целиком рядом**. Это удобно для точечных операций по ключу. Но у аналитических запросов другая форма: им нужно пройти миллионы строк и из каждой взять одну-две колонки.

На такой нагрузке строковая раскладка начинает тратить **I/O** впустую.

Когда строковая раскладка мешает

Возьмём запрос:

```
SELECT AVG(revenue) FROM sales;
```

База должна пройти все строки таблицы, но из каждой ей нужно ровно одно поле. Если `sales` весит **200 GB** и содержит 20 колонок, построчное хранение заставит прочитать все **200 GB** — хотя колонка `revenue` может занимать только **15 GB**.

Для таких запросов хочется разложить данные не по строкам, а **по колонкам**: все значения `revenue` подряд, все значения `country` подряд, и так далее.

Как выглядит колоночная раскладка

Возьмём маленькую таблицу `sales`:

id	date	product	country	revenue
----	-----	-----	-----	-----
1	2024-01-10	Laptop	USA	1200
2	2024-01-10	Phone	Germany	800
3	2024-01-11	Laptop	USA	1200
4	2024-01-11	Tablet	France	600
5	2024-01-12	Phone	Germany	800

Построчное хранение — как в PostgreSQL и InnoDB:

```
строка: [1, 2024-01-10, Laptop, USA, 1200][2, 2024-01-10, Phone, Germany, 800]...
```

Поколоночное хранение — как в ClickHouse, Vertica, Snowflake, форматах Parquet и ORC:

```
id:      [1, 2, 3, 4, 5]
date:    [2024-01-10, 2024-01-10, 2024-01-11, 2024-01-11, 2024-01-12]
product: [Laptop, Phone, Laptop, Tablet, Phone]
country: [USA, Germany, USA, France, Germany]
revenue: [1200, 800, 1200, 600, 800]
```

Разница на двух типичных запросах:

- `SELECT SUM(revenue) FROM sales` — строковая база читает всю таблицу; колоночная читает одну колонку `revenue`.
- `SELECT * FROM sales WHERE id = 3` — строковая база читает одну строку почти сразу; колоночная должна собрать её из нескольких колонок.

Отсюда главный обмен: **строковое хранение дешево отдаёт полную строку, колоночное дешево сканирует отдельные поля.**

На одной и той же таблице это видно и в памяти.

Сначала построчно:

id	date	product	country	rev
1	01-10	Laptop	US	12
2	01-10	Phone	DE	8
3	01-11	Laptop	US	12

Figure 1: Построчно: строки в памяти лежат подряд.

Теперь поколоночно:

id	date	product	country	rev
1	01-10	Laptop	US	12
2	01-10	Phone	DE	8
3	01-11	Laptop	US	12

Figure 2: Поколоночно: колонки в памяти лежат подряд.

Почему колонки хорошо сжимаются

Когда значения одной колонки лежат подряд, они однотипны и часто похожи друг на друга. Поэтому колоночный формат почти всегда идёт вместе с компрессией.

Типичные приёмы:

- **Run-Length Encoding (RLE)** — серии повторяющихся значений.
- **Dictionary Encoding** — словарь уникальных значений плюс короткие коды.
- **Bit Packing** — хранить число в минимальном числе бит.
- **Delta Encoding** — хранить не сами значения, а разности.
- **Frame of Reference (FOR)** — вычесть общий базовый уровень и упаковать остатки.

На длинных однородных колонках типичная степень сжатия — от 3х до 10х. Это важно не только для диска: чем меньше данных надо читать, тем быстрее сканирование.

Цена записи

У колоночной раскладки зеркальная проблема. Одна строка размазана по нескольким колонкам, поэтому точечные изменения получаются нелокальными:

- **INSERT** одной строки превращается в несколько записей;
- **UPDATE** одной ячейки затрагивает отдельный сегмент колонки;
- транзакция на одну строку перестаёт быть дешёвой и локальной.

Поэтому колоночные системы редко любят классический OLTP-стиль. Обычно они пишут **батчами**, используют append-only части и фоновые merge/compaction. В ClickHouse MergeTree это особенно видно: по духу это LSM, только над колонками.

OLTP vs OLAP

OLTP (online transaction processing):

- много коротких операций;
- чтение и запись отдельных строк;
- поддержка транзакций;
- поддержка constraints (FK, PK, UNIQUE итд);
- важна латентность одной транзакции;
- строковое хранение + heap или clustered + индексы.

Примеры систем: PostgreSQL, MySQL, Oracle.

OLAP (online analytical processing):

- редкие, но тяжёлые запросы;
- сканирование миллионов строк;
- агрегации по нескольким колонкам;
- важна пропускная способность скана;
- типичная физика: колоночное хранение + компрессия + запись батчами.

Примеры систем: ClickHouse, Vertica, Snowflake, DuckDB; форматы: Parquet, ORC.

Ключевая мысль: **OLTP и OLAP различаются не только по API и сценариям, а по физической раскладке данных.** Строковая раскладка делает дешёвыми точечные операции. Колоночная — большие сканы и агрегации.

Пример: маркетплейс

Один и тот же продукт почти всегда использует обе стороны.

OLTP-часть живёт в PostgreSQL или MySQL:

- добавить товар в корзину;
- оформить заказ;
- проверить остаток товара;
- обновить профиль пользователя.

OLAP-часть живёт в ClickHouse, Snowflake или DuckDB:

- посчитать конверсию по экранам;
- узнать, сколько пользователей iOS заходило в категорию за неделю;
- построить отчёт по выручке по странам;
- посчитать retention и воронки.

Типичный аналитический запрос:

```
SELECT COUNT(DISTINCT user_id)
FROM events
WHERE screen = 'category_electronics'
      AND os = 'iOS'
      AND timestamp > NOW() - INTERVAL '7 days'
```

При строковом хранении такой запрос тащит лишние поля каждой записи. При колоночном — читает в основном `screen`, `os`, `timestamp`, `user_id`, и поэтому живёт намного дешевле.